Azelmodsx677

NexusChat 8

Guía técnica para estudiar el código: cómo está construido, por qué, y cómo cambiarlo sin romperlo.



Versión del código: 8.0.0, commit 380298a · Noviembre de 2026

Cómo estudiar esta guía

Cada capítulo explica una parte y te indica qué archivos abrir. Las rutas son relativas a `app/src/main/java/com/Azelmods/App/`, salvo que se indique otra cosa. Las cifras se contaron en el propio repositorio y los fragmentos son de tu código, a veces abreviados.

1	Visión general y tecnologías	9	Tor y anonimato
2	Mapa de carpetas	10	Agentes de IA
3	Arquitectura por capas	11	Sistema de diseño
4	Arranque y navegación	12	Actualizaciones
5	Vida de un mensaje	13	Compilar y probar
6	Cifrado	14	Puntos a mejorar
7	Servidor: reglas y funciones	15	Ejercicios y glosario
8	Llamadas		

Orden recomendado

1. **Base:** capítulos 1 a 4. Sin esto, lo demás parece magia.
2. **El corazón:** capítulo 5, siguiendo un mensaje con el depurador.
3. **Tu sello:** capítulo 11. Es lo más fácil de modificar y lo que más se ve.
4. **Lo difícil:** capítulos 6, 8, 9 y 10, en el orden que te interese.
5. **Practica:** los ejercicios del capítulo 15.

> **Método:** lee la explicación, abre el archivo, pon un punto de ruptura en Android Studio y provoca la acción en el móvil. Ver los valores cambiar enseña más que leer.

Capítulo 1

Visión general y tecnologías

NexusChat es una app Android nativa cuyo servidor es Firebase: la app habla directamente con la base de datos, protegida por reglas, y unas pocas funciones en la nube hacen lo que el teléfono no puede, como enviar notificaciones o borrar lo caducado.

273 archivos Kotlin	**71.648** líneas Kotlin	**52** rutas de navegación	**35** archivos con ViewModel
30 nodos de base de datos	**13** funciones en la nube	**120** pruebas unitarias	**1.424** textos traducidos al inglés

Área	Tecnología	Para qué
Lenguaje e interfaz	Kotlin, Jetpack Compose, Material 3	Toda la interfaz es declarativa
Inyección	Hilt	Entrega repositorios y servicios a los ViewModel
Asincronía	Coroutines, StateFlow, callbackFlow	Trabajo en segundo plano y datos en vivo
Servidor	Firebase Auth, Realtime Database, Storage, FCM, Functions	Cuentas, datos, archivos, avisos y lógica de servidor
Local	Room, DataStore, EncryptedSharedPreferences	Caché, preferencias y secretos
Multimedia	Media3, Coil, Lottie	Vídeo, edición, imágenes y stickers animados
Llamadas	WebRTC (stream-webrtc-android)	Voz y vídeo entre teléfonos
Anonimato	tor-android, jtorctl	Tor integrado
Red e IA	OkHttp con SSE	Proveedores de IA con respuesta en directo

Requisitos: `minSdk 31` (Android 12) y `targetSdk 37`. El servidor usa Node.js 22.

Capítulo 2

Mapa de carpetas

Carpeta	Archivos	Líneas	Qué contiene
`ui/screens`	95	39.751	Pantallas y sus ViewModel, por función: chat, home, settings, stories...
`ui/components`	32	7.814	Piezas reutilizables: avatar, burbuja, stickers, `ChatLook`
`data/security`	25	3.668	Cifrado, Tor, modo fantasma, bloqueo, TOTP
`data/ai`	15	3.552	Proveedores de IA y motor de agentes
`data/repository`	9	3.264	Acceso a Firebase y Storage
`data/backup`	4	1.180	Copias de seguridad cifradas
`service` , `services`	7	1.273	Notificaciones (FCM) y servicio de llamada
`webrtc`	4	824	Conexión de llamadas, audio y servidores ICE
`ui/theme`	7	819	Colores, tipografía y tokens de diseño
`data/local`	11	699	Room: 4 tablas, 4 DAO y `CacheManager`
`ui/navigation`	3	636	Rutas y grafo de navegación
`di`	4	271	Módulos de Hilt

Fuera de la app: `functions/index.js` (1.029 líneas de servidor), `database.rules.json` y `storage.rules` (permisos), y `app/src/test` (14 archivos de prueba).

Los archivos más grandes

Archivo	Líneas	Por qué importa
`RealtimeDatabaseRepository.kt`	1.764	Casi toda la comunicación con la base de datos
`ChatViewModel.kt`	1.711	Toda la lógica de una conversación
`AgentsPanel.kt`	1.209	Interfaz de los agentes
`ChatScreen.kt`	1.174	Pantalla de chat
`HomeScreenRedesigned.kt`	1.080	Lista de chats
`ChatComposer.kt`	1.077	Campo de escribir y sus 8 estilos

Capítulo 3

Arquitectura por capas

La app sigue MVVM: la pantalla solo dibuja, el ViewModel decide y el repositorio trae los datos. La regla es que cada capa solo conoce a la de abajo.



Las órdenes bajan; los datos suben como flujos que Compose observa.

Ejemplo real: la lista de chats sin conexión

`HomeViewModel` enseña primero lo guardado y luego escucha a Firebase. Por eso la lista aparece al instante incluso sin red.

```
// ui/screens/home/HomeViewModel.kt (abreviado)
val cachedEntities = cacheManager.getCachedChats()      // 1. Room: inmediato
if (cachedEntities.isNotEmpty()) _state.value = ...       // 2. se muestra ya
databaseRepository.getUserChats(userId)                  // 3. Firebase en vivo
    .debounce(120L)                                      // espera a que lleguen todos los campos
    .collectLatest { rawChats ->
        val enriched = rawChats.mapNotNull { buildChatFromMap(it, userId) }
        cacheManager.replaceChats(enriched)              // 4. actualiza la caché
        _state.value = _state.value.copy(chats = enriched)
    }
```

Inyección con Hilt

Los ViewModel no crean sus dependencias; las piden en el constructor. `di/AppModule.kt`, `FirebaseModule.kt`, `RepositoryModule.kt` y `SecurityModule.kt` saben fabricarlas. En una prueba puedes entregar una versión falsa, como hacen las pruebas de `ChatViewModel`.

Capítulo 4

Arranque y navegación

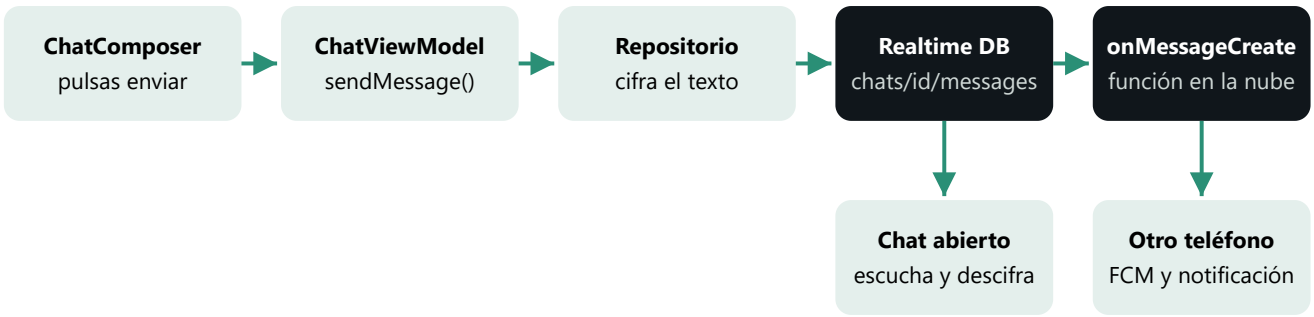
1. `MainActivity` arranca (es `FragmentActivity` por el diálogo de huella) e inicia `ChatLook` y `ChatOrganizer`, que leen tus preferencias.
2. `NavGraph.kt` define las 52 rutas con `composable(...)`. `Screen.kt` les da nombre para no escribir textos sueltos.
3. Tras la sesión se llega a `MainScreen`: un `HorizontalPager` con Chats, Historias, Llamadas y Perfil, más la barra inferior.

```
// ui/screens/main/MainScreen.kt: cada página del pager y su pantalla
when (tabOrder.getOrNull(page) ?: page) {
    0 -> HomeScreenRedesigned(navController = navController, onOpenTab = { ... })
    1 -> StoriesScreen(navController = navController)
    2 -> CallsScreen(navController = navController)
    3 -> ProfileScreen(navController = navController, userId = userId, showBack = false)
}
```

Hay dos niveles: el **pager** cambia de pestaña sin perder el estado, y el **NavController** abre pantallas encima, como un chat o Ajustes. Para añadir una pantalla basta registrar la ruta en `NavGraph.kt` y llamar a `navController.navigate("ruta")`.

Capítulo 5

Vida de un mensaje



Abre `sendMessageInternal` en `RealtimeDatabaseRepository.kt`. Si la lectura del chat falla, no asume que existe: sigue al bloque de creación, que fusiona datos en vez de reemplazarlos. Es un buen ejemplo de código defensivo.

Cómo se escucha la lista de chats

`getUserChats` es la mejor lección sobre Realtime Database. El índice `userChats/{uid}` dice en qué chats estás. Por cada uno escucha solo los campos de vista previa, nunca los mensajes, porque descargar `chats/{id}` entero traería toda la conversación.

```
fun getUserChats(userId: String) = callbackFlow {
    fun watch(chatId: String) {
        pendingFirstLoads += CHAT_PREVIEW_FIELDS.size
        watchers[chatId] = CHAT_PREVIEW_FIELDS.map { field ->
            val ref = database.child("chats").child(chatId).child(field)
            ref.addValueEventListener(listener); ref to listener
        }
    }
    val indexRef = database.child("userChats").child(userId)
    indexRef.addValueEventListener(indexListener)  // añade y quita chats
    awaitClose { /* retira todos los listeners: evita fugas */ }
}
```

Fíjate en tres ideas: `pendingFirstLoads` impide publicar una lista a medias, `callbackFlow` convierte callbacks en un flujo, y `awaitClose` libera todo al salir de la pantalla.

Datos en Realtime Database

Nodo	Contenido
`users`, `fcmTokens`	Perfiles, claves públicas y tokens de notificación
`userChats`, `chats`	Índice por usuario, y chats con sus miembros y mensajes
`groups`, `groupKeys`, `groupInvites`	Grupos, sus claves cifradas para cada miembro e invitaciones
`secretChats`, `userSecretChats`	Buzón temporal de chats secretos
`stories`, `calls`, `userCalls`	Historias, llamadas e historial
`channels`, `channelPosts`	Canales y publicaciones
`expiringMessages`	Lista de lo que debe borrarse y cuándo
`config`	Configuración remota, como la actualización

Capítulo 6

Cifrado

Tienes tres sistemas, cada uno con un equilibrio distinto entre comodidad y secreto.

	Chat normal	Grupo	Chat secreto
Clave	ECDH entre claves fijas de identidad	Una AES-256 aleatoria por grupo	ECDH efímero; una clave por mensaje
Cifrado	AES-256-GCM	AES-256-GCM	AES-256-GCM con datos asociados
Archivo	`E2EECryptoService.kt`	`GroupCrypto.kt`	`SecretRatchet.kt`, `SecretChatRepository.kt`
Secreto hacia adelante	No	Solo al cambiar la clave	Sí, avanza y olvida
Copia en servidor	Sí, cifrada	Sí, cifrada	Hasta entregarse

Chat de dos personas

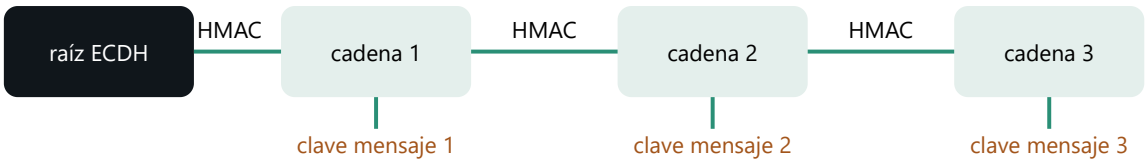
```
// data/security/encryption/E2EECryptoService.kt
val agreement = KeyAgreement.getInstance("ECDH")
agreement.init(local.private)               // mi clave privada
agreement.doPhase(remotePublic, true)        // la pública del otro
val key = MessageDigest.getInstance("SHA-256").digest(agreement.generateSecret())

val cipher = Cipher.getInstance("AES/GCM/NoPadding")
cipher.init(Cipher.ENCRYPT_MODE, SecretKeySpec(aesKey, "AES"))
val encrypted = cipher.doFinal(plaintext.toByteArray())   // se envía iv + encrypted
```

La idea de ECDH: tu privada con su pública da el mismo secreto que su privada con tu pública, sin que ese secreto viaje nunca. GCM añade una etiqueta que delata cualquier alteración. Las privadas se guardan por uid en `EncryptedSharedPreferences` y las públicas se publican en `users/{uid}/keys`. `safetyNumber()` permite comparar claves entre dos personas para detectar una sustitución.

> El comentario de esta clase explica un fallo real: antes las claves eran únicas por dispositivo, y con dos cuentas el segundo usuario reutilizaba la privada del primero, así que nada se descifraba. Por eso ahora son por uid.

Chat secreto: la cadena que olvida



HMAC no se puede invertir: quien robe la cadena 3 no obtiene las claves 1 y 2.

```
fun advance(chain: ByteArray) = Step(hmac(chain, MESSAGE_KEY_LABEL), hmac(chain, NEXT_CHAIN_LABEL))
```

Cada participante tiene su cadena (`chainFor`), de modo que nunca se reutiliza una clave. `seal` liga además cada mensaje a su chat, remitente y número mediante datos asociados.

Grupos y copias

`GroupCrypto` crea una clave aleatoria y la cifra por separado para cada miembro en `groupKeys/{chatId}/{keyId}/{uid}`. Si cambian los miembros se crea otra, y quien salió no puede leer lo siguiente. `BackupEncryptor` deriva la clave de tu contraseña con PBKDF2 (100.000 iteraciones) y usa AES-256-GCM con HMAC-SHA256.

Capítulo 7

Servidor: reglas y funciones

Como la app habla directamente con la base de datos, **las reglas son tu verdadera seguridad**. Cualquiera puede modificar una app; las reglas se ejecutan en el servidor.

```
// database.rules.json
"chats": { "$chatId": {
  ".read":  "auth != null && (!data.exists() || data.child('members').child(auth.uid).exists())",
  ".write": "auth != null && (!data.exists() || (... miembro y sin bloqueo ...))"
}}
```

> En Realtime Database **un permiso concedido por el padre no se puede retirar en un hijo**. Por eso el bloqueo se comprueba en `chats/$chatId` y no en cada mensaje. Es el error de seguridad más común en Firebase.

Funciones en la nube (`functions/index.js`)

Función	Se activa	Hace
`onMessageCreate`	Mensaje nuevo	Avisa a los otros miembros por FCM
`onSecretMessage`	Mensaje secreto	Avisa sin revelar contenido
`onStoryCreate`, `onStoryReaction`	Historia o reacción	Avisa a los contactos o al autor
`onCallCreate`, `onCallEnded`	Cambios en `calls`	Timbre y llamada perdida
`onChannelPost`, `onChannelSubscriber`	Canales	Publicaciones y suscriptores
`indexEphemeralMessage`	Mensaje temporal	Lo apunta en `expiringMessages`
`cleanupExpiredMessages`	Cada minuto	Borra lo caducado, hasta 500 cada vez
`cleanupExpiredStories`	Programada	Borra historias vencidas
`checkAppRelease`	Cada 15 minutos	Detecta releases nuevos de GitHub
`premiumApi`	Petición HTTP	Pagos de Premium; pendiente de desplegar

Capítulo 8

Llamadas

WebRTC conecta los dos teléfonos directamente; Firebase solo presenta a las partes (**señalización**).

1. Quien llama crea una oferta SDP (`createOffer` en `webrtc/WebRTCManager.kt`) y la escribe en `calls`.
2. `onCallCreate` envía el aviso y el otro teléfono muestra la llamada (`showIncomingCallNotification`).
3. Al aceptar se responde, y ambos intercambian candidatos ICE: las rutas de red posibles.
4. La media fluye directa; `CallService` mantiene viva la llamada en segundo plano.

`IceServerProvider.kt` entrega los servidores. Un STUN solo descubre tu dirección pública; en redes restrictivas hace falta un TURN que retransmita.

Capítulo 9

Tor y anonimato

Vive en `data/security/tor/`, con 16 archivos.

Archivo	Función
`EmbeddedTor.kt`	Arranca el Tor incluido, sin instalar nada
`TorService.kt`, `TorState.kt`	Ciclo de vida y estado de la conexión
`OrbotDetector.kt`, `OrbotIntents.kt`	Integración con Orbot si lo tienes
`FirebaseProxyConfigurator.kt`	Lleva el tráfico de Firebase por Tor
`TorDnsResolver.kt`	Resuelve DNS por Tor para evitar fugas
`TorConnectivityChecker.kt`	Comprueba que la salida es de verdad por Tor
`AnonymityPolicy.kt`	Decide qué se permite en modo anónimo

> La lección es **fallar cerrado**: si el modo anónimo está activo y Tor cae, es mejor no conectar que filtrar tu IP. Estudia `AnonymityPolicy` y `TorSocketRouter` con esa pregunta en mente.

Capítulo 10

Agentes de IA

El motor (`data/ai/agents/AgentOrchestrator.kt`, 512 líneas) sigue el patrón **ReAct**: pensar, usar una herramienta, observar y repetir hasta tener la respuesta.



Archivo	Papel
`AgentCatalog.kt`	Agentes y equipos, con sus instrucciones
`AgentProtocol.kt`	Protocolo JSON propio, en vez del tool calling de cada proveedor
`AgentToolbox.kt`	Las herramientas reales (919 líneas)
`JsSandbox.kt`	Ejecuta JavaScript en un WebView aislado
`SafeCalculator.kt`	Aritmética exacta: los modelos se equivocan
`WebSearch.kt`	Búsqueda en internet

> **Inyección de instrucciones:** un agente lee páginas de internet, y esas páginas pueden intentar darle órdenes. Por eso el JavaScript va aislado y guardar archivos pide tu permiso. Todo lo que venga de fuera es dato, no instrucción.

Capítulo 11

Sistema de diseño

Toda la personalización se apoya en `ui/components/ChatLook.kt`: un objeto con un `StateFlow` por opción, guardado en preferencias. Al cambiar el valor, cada pantalla que lo observa se redibuja sola.

Receta: añadir una opción

Así se añadió el juego de iconos de la barra:

```
// 1. Las opciones
enum class NavIconSet(val title: String) { OUTLINED("Contorno"), FILLED("Relleno") }
// 2. Estado y guardado en ChatLook
private val _navIconSet = MutableStateFlow(NavIconSet.OUTLINED)
val navIconSet: StateFlow<NavIconSet> = _navIconSet.asStateFlow()
fun setNavIconSet(set: NavIconSet) {
    _navIconSet.value = set
    prefs?.edit()?.putString("nav_icons", set.name)?.apply()
}
// 3. Leerlo al arrancar, en init()
// 4. Control en ChatStudioScreen.kt
ChoiceChips(NavIconSet.entries, navIcons, { it.title }, ChatLook::setNavIconSet)
// 5. Usarlo en la pantalla
val navIconSet by ChatLook.navIconSet.collectAsState()
```

Y añade el texto a `assets/i18n/en.json`, porque `tr("...")` traduce buscando la frase en español.

Dibujar a mano: el aro central

El aro de la barra se dibuja con `drawBehind`. Sus proporciones y su ritmo se tomaron del análisis de `ImgView` en V2: seis arcos de 48° y un giro cada 2 segundos.

```
val angle by spin.animateFloat(0f, 360f, infiniteRepeatable(tween(2_000, easing = LinearEasing)))
Modifier.drawBehind {
    val paint = Brush.linearGradient(listOf(accent, Color.White))
    rotate(angle) {
        repeat(6) { segment -> drawArc(paint, segment * 60f, 48f, false, topLeft, arcSize, style =
stroke) }
    }
}
```

El color: `AccentState` es estado de Compose, y `NexusTokens.Color.Primary` lo lee, así que cambiar el acento repinta toda la app. Usa siempre tokens de `ui/theme/`, no colores fijos.

Capítulo 12

Actualizaciones

1. `checkAppRelease` consulta GitHub cada 15 minutos.
2. Con una versión nueva escribe `config/appUpdate` y avisa por FCM. `mandatory` es verdadero salvo que las notas lleven `[opcional]`.
3. `AppUpdateManager` escucha ese nodo, y `isNewerVersion` compara versiones.
4. `apkUrlFor` elige el APK por arquitectura o el universal, y solo acepta enlaces de confianza (`TRUSTED_DOWNLOADS`).
5. `AppUpdateDialog` no se puede cerrar si es obligatoria.

Capítulo 13

Compilar y probar

```
# APK de prueba
./gradlew :app:assembleDebug
# Pruebas unitarias (120)
./gradlew :app:testDebugUnitTest
# Análisis estático
./gradlew :app:lintDebug
# APK firmado, con R8; necesita keystore.properties, que nunca se sube
./gradlew :app:assembleRelease
# Servidor, con Node 22
npx -p node@22 -p firebase-tools firebase deploy --only database,storage,functions
```

Las pruebas usan JUnit y Kotest. El motor `junit-vintage-engine` es necesario para ejecutar las de JUnit 4: sin él pasaban por alto sin avisar. El release activa `isMinifyEnabled` (R8) y genera un APK por arquitectura más el universal.

Capítulo 14

Puntos a mejorar

Toda app de este tamaño tiene deuda técnica. Saber verla es parte de estudiarla.

Punto	Por qué importa	Qué haría
Chats normales sin secreto hacia adelante	Si roban la clave privada, se pueden leer los mensajes pasados	Llevar el ratchet de los chats secretos a todos
Cifrado sin auditoría externa	Los errores de criptografía son invisibles para las pruebas normales	Publicar el diseño y pedir una revisión
SHA-256 sobre el secreto ECDH en chats normales	HKDF, como en los secretos, es la práctica estándar	Unificar con HKDF; requiere migración
Logs con parte del contenido	`sendMessage` registra el inicio del texto antes de cifrar, y no hay regla R8 que retire los `Log`	Quitarlo y usar `-assumenosideeffects`
Archivos gigantes	`RealtimeDatabaseRepository` y `ChatViewModel` son difíciles de probar	Dividir por dominio: grupos, temporales, bloqueo
Carpetas duplicadas	`service` y `services`; `util` y `utils`	Unificarlas
Licencia	La pantalla dice derechos reservados; el repositorio, MIT; y hay librerías GPL	Alinear el texto y revisar esas dependencias

Capítulo 15

Ejercicios y glosario

Ejercicios, de fácil a difícil

1. **Una opción propia.** Añade un interruptor al Estudio siguiendo la receta del capítulo 11.
2. **Otra barra.** Crea una novena barra en `MainScreen.kt` con un icono que crezca al seleccionarlo.
3. **Sigue un mensaje.** Pon puntos de ruptura en `sendMessage`, `sendMessageInternal` y `encryptFor`. Mira el valor cifrado en la consola de Firebase.
4. **Limpia los logs.** Busca los `Log.d` que muestran contenido y elimínalos.
5. **Divide el repositorio.** Saca el bloqueo de `RealtimeDatabaseRepository` a su propia clase sin romper las pruebas.
6. **Escribe una regla.** Prueba en el simulador de Firebase si un no miembro puede leer un chat.
7. **Mejora la cripto.** Escribe pruebas del ratchet: la misma cadena debe dar siempre las mismas claves, y un mensaje alterado debe fallar.
8. **Una herramienta para agentes.** Añade una herramienta simple en `AgentToolbox.kt` y descríbela en `AgentModels.kt`.

Glosario

Composable	Función que describe interfaz y se redibuja cuando cambia su estado
StateFlow	Valor observable que siempre tiene un estado actual
callbackFlow	Convierte una API de callbacks en un flujo de corrutinas
ViewModel	Guarda el estado de una pantalla y sobrevive a girarla
Hilt	Fabrica y entrega las dependencias de cada clase
Room	Base de datos SQLite local con comprobación al compilar
ECDH	Acordar un secreto común intercambiando solo claves públicas
AES-GCM	Cifrado rápido que además detecta alteraciones
HMAC / HKDF	Resumen con clave / derivación de claves a partir de un secreto
Secreto hacia adelante	Robar la clave de hoy no descifra los mensajes de ayer
SDP / ICE	Descripción de una llamada / búsqueda de la ruta de red
STUN / TURN	Descubre tu dirección pública / retransmite si no hay ruta directa
FCM	Servicio de Google para despertar el teléfono con un aviso
R8	Reduce y ofusca el APK de release
ReAct	Patrón de agente: razonar, actuar y observar en bucle